Linear regression

Định giá căn hộ từ diện tích và khoảng cách

Phase 2 · Bài 2.1

Hời thật không?

Một căn hộ **70 m²**, cách trung tâm **8 km**, rao bán **4,2 tỷ**.

“Giá này rất hời.”

Để thử cách trả lời, buổi học dựng bảng **200 căn giả lập** với diện tích, khoảng cách và giá. Cần so dự đoán với giá rao như thế nào?

TRỰC GIÁC

Căn rộng hơn thì đắt hơn. Xa trung tâm thì rẻ đi.

Linear regression là trực giác này được viết thành toán học.

Mục tiêu sau buổi này

- Hiểu mô hình tuyến tính giả định điều gì và đọc được ý nghĩa từng hệ số.
- Tìm nghiệm bằng hai cách: nghiệm đóng và gradient descent.
- Giải thích vì sao learning rate lớn thì loss nổ, và vì sao phải chuẩn hóa đặc trưng.
- Dùng scikit-learn đúng quy trình và tránh các bẫy cài đặt hay gặp.

01

Dữ liệu và mô hình

Mô hình tuyến tính

Mỗi đặc trưng đóng góp một lượng tỉ lệ với nó, rồi cộng lại:

$$\hat{y} = w_1 x_1 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b$$

w_j: dự đoán tăng bao nhiêu khi x_j tăng 1 đơn vị **và các đặc trưng khác giữ nguyên**.

Cho cả n mẫu cùng lúc:

$$\hat{\mathbf{y}} = X\mathbf{w} + b$$

Ký hiệu	Shape
X	`(n, d)`
$\mathbf{w}$	`(d,)`
$\hat{\mathbf{y}}$, $\mathbf{y}$	`(n,)`

Dữ liệu của buổi học

Tự sinh 200 căn theo một công thức biết trước, cộng nhiễu:

$$\text{giá} = 0.05 \cdot \text{diện tích} - 0.15 \cdot \text{khoảng cách} + 1$$

Biết trước đáp án thì kiểm tra được mô hình học đúng hay sai.

```
rng = np.random.default_rng(0)
n = 200
area = rng.uniform(30, 120, n)
dist = rng.uniform(1, 15, n)
y = 0.05 * area - 0.15 * dist + 1.0 \
    + rng.normal(0, 0.3, n)
X = np.column_stack([area, dist])
print(X.shape, y.shape)
```

KẾT QUẢ

```
(200, 2) (200,)
```

HỎI NHANH

`X` có shape `(200, 2)`, `w` có shape `(2,)`. `X @ w + b` có shape gì?

A `(200,)`

B `(2,)`

C `(200, 2)`

D `(200, 1)`

02

Hàm mất mát

Mean squared error

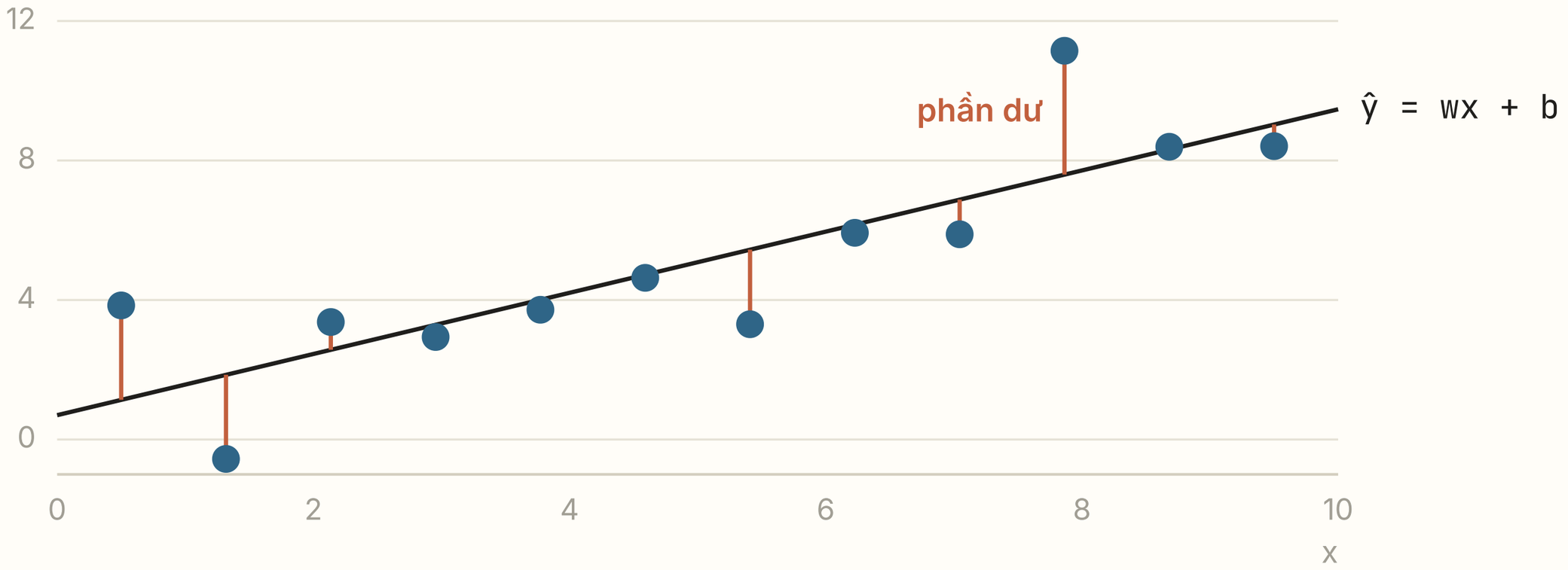
$$J(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

Vì sao bình phương?

- Trơn, đạo hàm được ở mọi điểm.
- Phạt lỗi lớn nặng hơn: sai 10 bị phạt gấp 4 lần sai 5.
- Nếu phần nhiễu độc lập, trung bình bằng 0 và có cùng phương sai theo điều kiện trên đặc trưng, MSE tương ứng với maximum likelihood dưới giả định Gaussian.

Nhược điểm: nhạy với outlier.

Phần dư trên một đường thẳng



Phần dư là đoạn thẳng đứng từ điểm dữ liệu tới đường dự đoán.

Mốc so sánh

Mô hình đoán mọi căn bằng giá trung bình có

$$\mathrm{MSE} = \mathrm{Var}(y) = 2.34$$

Trên cùng tập chấm, mô hình có MSE lớn hơn mốc này chưa cải thiện sai số so với dự đoán hằng số.

$$R^2 = 1 - \frac{\mathrm{MSE}}{\mathrm{Var}(y)}$$

$R^2 = 0$: ngang mốc · $R^2 = 1$: đoán trúng hết

HỎI NHANH

Đổi đơn vị giá từ tỷ đồng sang triệu đồng, tức nhân mọi y với 1000. MSE thay đổi thế nào?

A Không đổi

B Tăng 1000 lần

C Tăng 10^6 lần

D Giảm 1000 lần

03

Nghiệm đóng

Phương trình chuẩn

Thêm cột 1 vào X: $\tilde{X} = [X \;\; \mathbf{1}]$, gộp $\boldsymbol{\theta} = (\mathbf{w}, b)$, khi đó $\hat{\mathbf{y}} = \tilde{X}\boldsymbol{\theta}$.

Cực tiểu của J là nơi gradient bằng 0:

$$\nabla J(\boldsymbol{\theta}) = \frac{2}{n}\tilde{X}^\top(\tilde{X}\boldsymbol{\theta} - \mathbf{y}) = \mathbf{0} \quad \Longleftrightarrow \quad \tilde{X}^\top\tilde{X}\,\boldsymbol{\theta} = \tilde{X}^\top\mathbf{y}$$

Nếu các cột của $\tilde{X}$ độc lập, hệ có nghiệm duy nhất và có thể giải bằng `np.linalg.solve`. Với hệ suy biến hoặc điều kiện kém, dùng `lstsq`/QR/SVD thay cho nghịch đảo ma trận.

Giải trên dữ liệu căn hộ

```
Xb = np.column_stack([X, np.ones(n)])
theta = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)
w_cf, b_cf = theta[:2], theta[2]
print(w_cf.round(4), round(b_cf, 4))
print(mse(predict(X, w_cf, b_cf), y))
print(predict(np.array([70, 8]), w_cf, b_cf))
```

KẾT QUẢ

```
[ 0.0499 -0.1453] 0.9809
0.08807615180405967
3.309815169606732
```

- Hệ số $\approx (0.05,\ -0.15,\ 1)$: khôi phục đúng công thức sinh dữ liệu.
- MSE $0.088 \approx 0.3^2$: nhiễu độc lập không dự đoán được cho căn mới; train noise vẫn có thể bị học thuộc.
- Căn 70 m², 8 km: mô hình định giá **3,3 tỷ**. Rao 4,2 tỷ là cao hơn 0,9 tỷ.

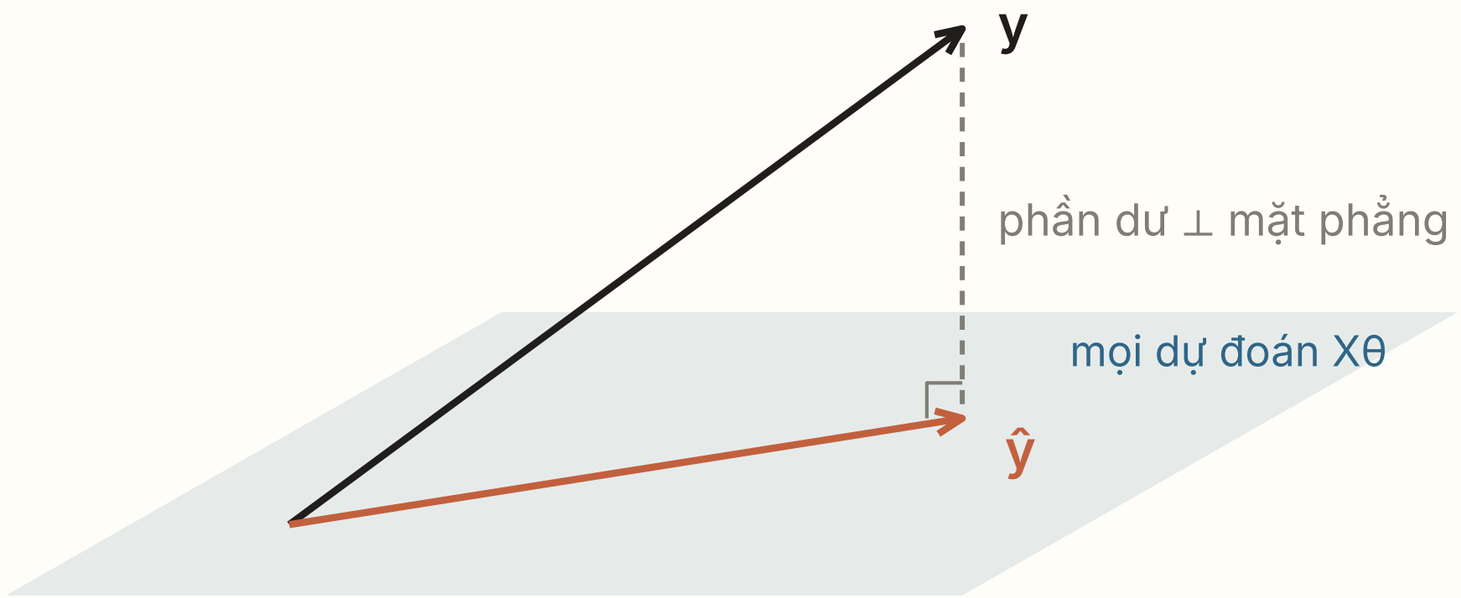
Cách nhìn hình học

$\mathbf{y}$ là một vector trong không gian n chiều. Mọi dự đoán $\tilde{X}\boldsymbol{\theta}$ nằm trên mặt phẳng sinh bởi các cột của $\tilde{X}$.

Least squares chọn **hình chiếu vuông góc** của $\mathbf{y}$ lên mặt phẳng đó.

Phần dư vuông góc với mọi cột, kể cả cột số 1, nên tổng phần dư bằng 0.

Hình chiếu vuông góc



Least squares chiếu y vuông góc xuống mặt phẳng các dự đoán; phần dư vuông góc với mọi cột.

HỎI NHANH

Thêm vào X một cột “diện tích tính bằng dm²”, đúng bằng 100 lần cột diện tích. Nghiệm đóng sẽ ra sao?

A Chính xác hơn, vì có thêm một cột thông tin

B Không đổi gì, cột mới chỉ nhận hệ số 0

C Ma trận suy biến, hệ số có thể rất lớn và trái dấu

D Chỉ chạy chậm hơn một chút vì thêm một cột

04

Gradient descent

Đi xuống trong sương mù

Không thấy đáy, chỉ cảm nhận độ dốc dưới chân: bước một bước nhỏ về phía dốc xuống, rồi lặp lại.

Với $r_i = \hat{y}_i - y_i$:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{2}{n} X^\top \mathbf{r}, \qquad \frac{\partial J}{\partial b} = \frac{2}{n} \sum_i r_i$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial J}{\partial \mathbf{w}}, \qquad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

```
def grad(theta, X, y):
    r = predict(X, theta[:-1], theta[-1]) - y
    return np.append(2 / len(y) * X.T @ r,
                     2 * r.mean())


def gd(X, y, lr, epochs):
    theta = np.zeros(X.shape[1] + 1)
    history = []
    for _ in range(epochs):
        history.append(loss(theta, X, y))
        theta -= lr * grad(theta, X, y)
    return theta, history
```

HỎI NHANH

Đặt `r = y_hat - y`. Dòng nào cập nhật bias đúng?

A `b -= lr * 2 * r.mean()`

B `b += lr * 2 * r.mean()`

C `b -= lr * 2 * r.sum()`

D `b -= lr * r`

Tính tay trên lớp

Ba điểm $(1, 2)$, $(2, 3)$, $(3, 5)$.

1. Tìm đường least squares $\hat{y} = wx + b$.
2. Từ $w = b = 0$, $\eta = 0.1$, làm một bước gradient descent.

Công thức cho một đặc trưng:

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \qquad b = \bar{y} - w\bar{x}$$

Lời giải

$\bar{x} = 2$, $\bar{y} = 10/3$, tử số $= 3$, mẫu số $= 2$:

$$w = 1.5, \qquad b = \tfrac{1}{3}, \qquad J = 0.056$$

Tại $w = b = 0$: $r = (-2, -3, -5)$,

$$\frac{\partial J}{\partial w} = -\frac{46}{3}, \qquad \frac{\partial J}{\partial b} = -\frac{20}{3}$$

Sau một bước: $w = 1.53$, $b = 0.67$, loss từ 12.67 xuống 0.22.

Kiểm tra gradient

Xấp xỉ đạo hàm chỉ bằng hàm loss, như viết code trâu để đối chiếu:

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\boldsymbol{\theta} + h\mathbf{e}_j) - J(\boldsymbol{\theta} - h\mathbf{e}_j)}{2h}$$

Sai số $O(h^2)$, chọn $h \approx 10^{-5}$.

Phiên bản `grad`	Sai số tương đối
Đúng	$7 \cdot 10^{-13}$
Quên hệ số 2	0.333

Sai số nhỏ xác nhận đạo hàm tại điểm đã kiểm. Kiểm vài điểm và thang bước sai phân; bước quá nhỏ có thể tăng sai số số học.

HỎI NHANH

Vì sao ta dùng sai phân trung tâm $\frac{J(\theta+h)-J(\theta-h)}{2h}$ thay vì $\frac{J(\theta+h)-J(\theta)}{h}$?

A Vì tính nhanh hơn

B Vì sai số là $O(h^2)$ thay vì $O(h)$

C Vì không phải chọn h

D Vì sai phân một phía có thể ra số âm

05

Learning rate và chuẩn hóa

Thử ba learning rate

```
for lr in [1e-3, 2e-4, 1e-4]:
    theta_gd, hist = gd(X, y, lr, 1000)
    print(f"lr={lr:g}, loss={hist[-1]:.4g}")
```

KẾT QUẢ

```
lr=0.001, loss=nan
lr=0.0002, loss=inf
lr=0.0001, loss=0.1541
```

- Hai giá trị đầu: loss nổ tung.
- 10^{-4}: hội tụ nhưng rất chậm; sau 1000 vòng, b mới đi được 0.01 trên đường tới 0.98.

Có đoán trước được không?

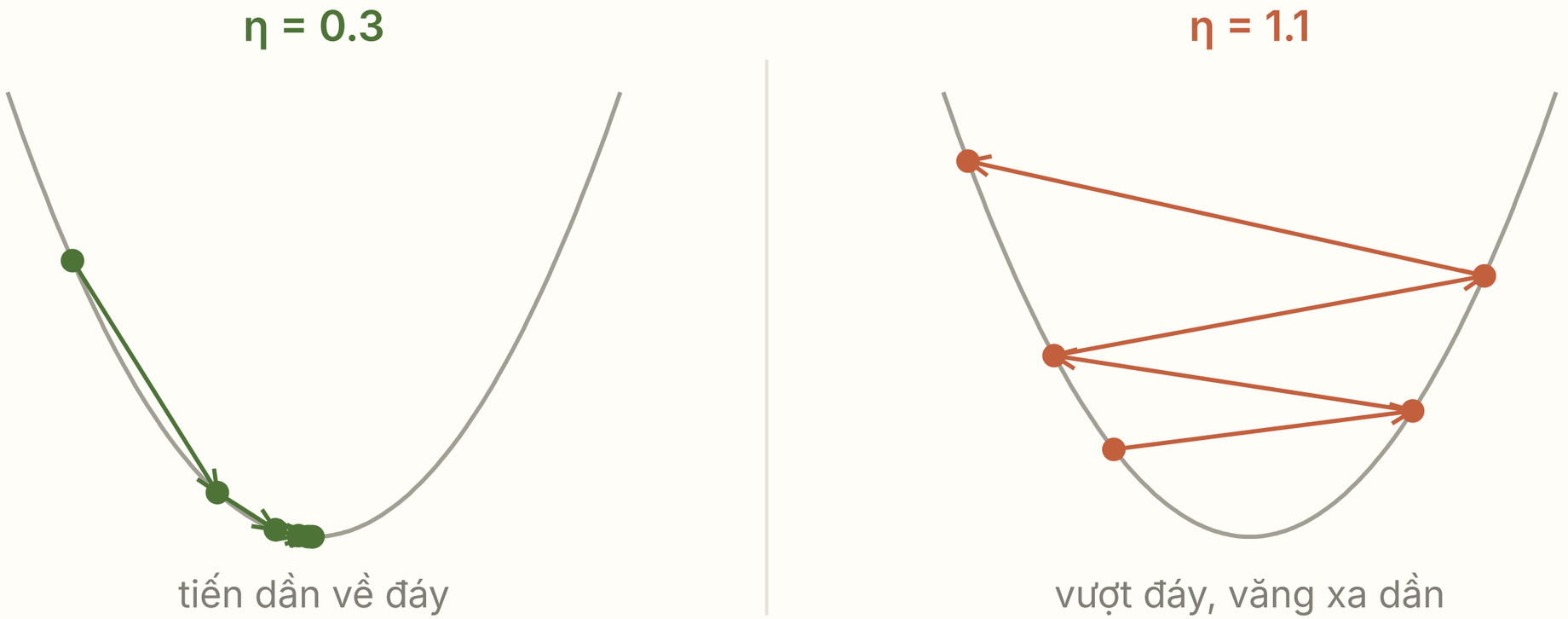
Ngưỡng của learning rate

Hessian của J là hằng số $H = \frac{2}{n}\tilde{X}^\top \tilde{X}$. Mỗi bước nhân sai số theo hướng ứng với trị riêng λ với $(1 - \eta\lambda)$, nên cần

$$0 < \eta < \frac{2}{\lambda_{\max}}$$

Dữ liệu căn hộ: ngưỡng $= 1.43 \cdot 10^{-4}$, số điều kiện $\kappa = \lambda_{\max}/\lambda_{\min} \approx 10^5$.

Bước vừa phải và bước quá lớn



Bước vừa phải đi dần về đáy; bước quá lớn vượt đáy rồi văng xa dần.

HỎI NHANH

Linear regression tối ưu MSE, loss qua các vòng là 15.9, 41.2, 260, 17 000, rồi NaN. Shape và công thức gradient đều đã kiểm. Ta nên thử gì trước?

A Tăng learning rate để vượt qua vùng loss cao

B Giữ learning rate, chỉ đổi seed vài lần

C Giảm learning rate, và kiểm thang đo của các feature

D Tăng số vòng để loss có thời gian ổn định

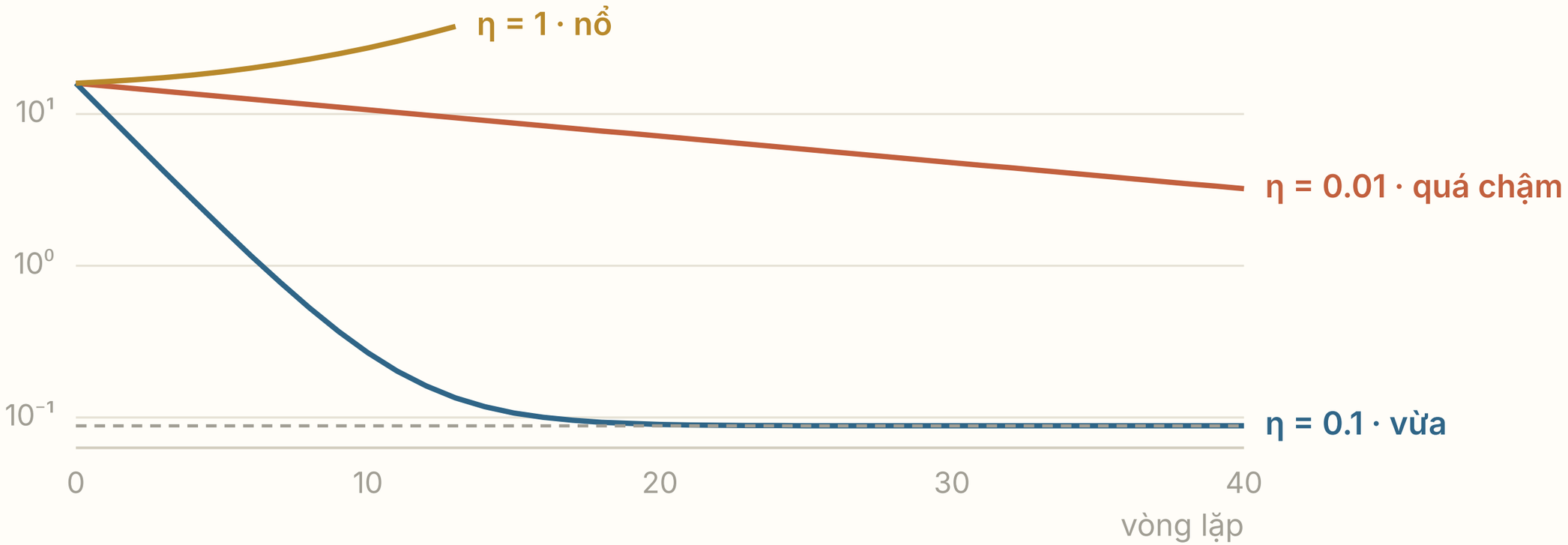
Chuẩn hóa đặc trưng

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

	Dữ liệu gốc	Đã chuẩn hóa
Số điều kiện	10^5	1.1
Ngưỡng η	$1.4 \cdot 10^{-4}$	0.95
Số vòng tới sát đáy	~180 000	28

μ, σ tính trên tập train, dùng lại cho test.

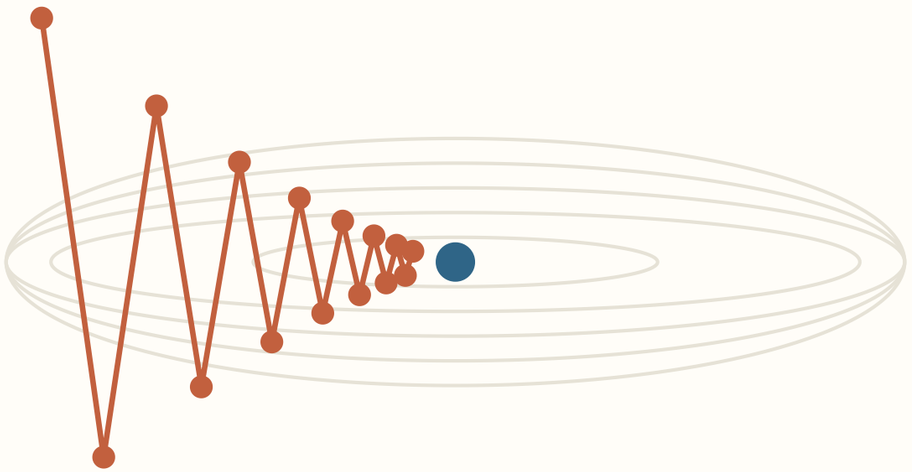
Sau chuẩn hóa: ba learning rate



Loss theo vòng lặp (thang log) với ba learning rate.

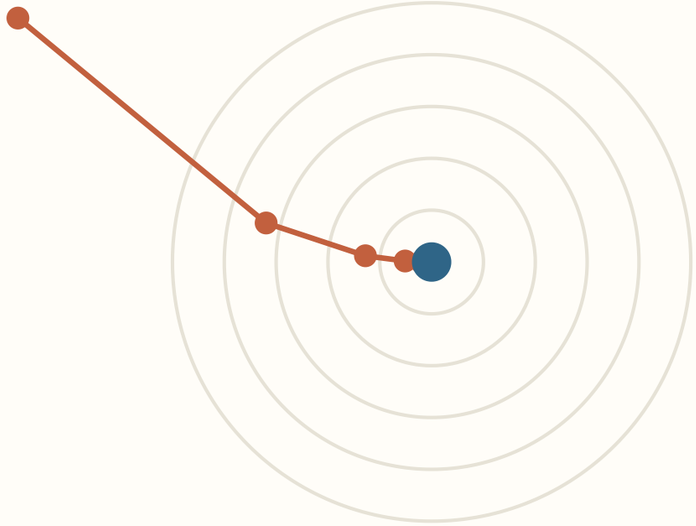
Thung lũng hẹp thành cái bát tròn

chưa chuẩn hóa



zigzag trong thung lũng hẹp

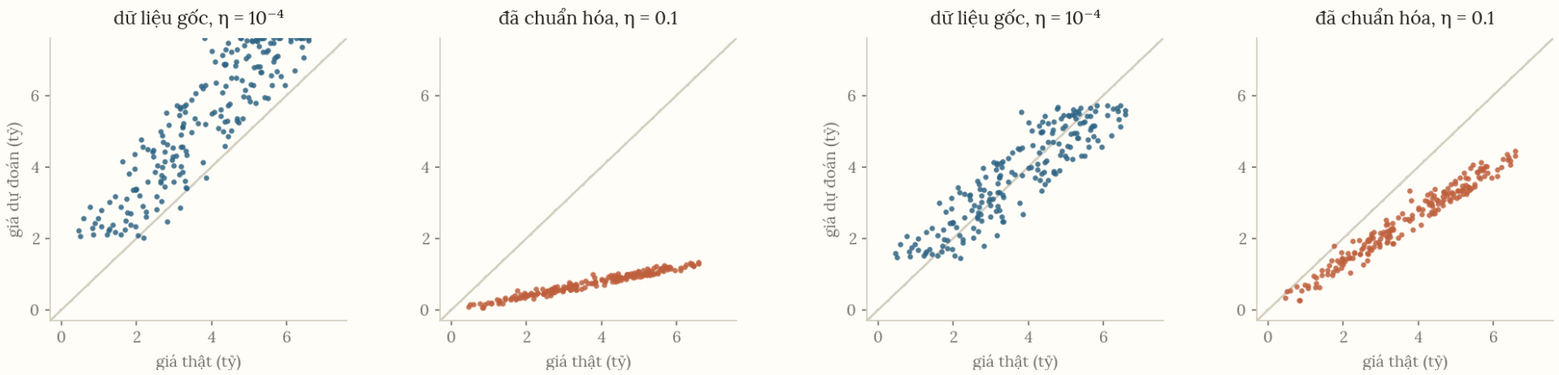
đã chuẩn hóa



đi gần thẳng về đáy

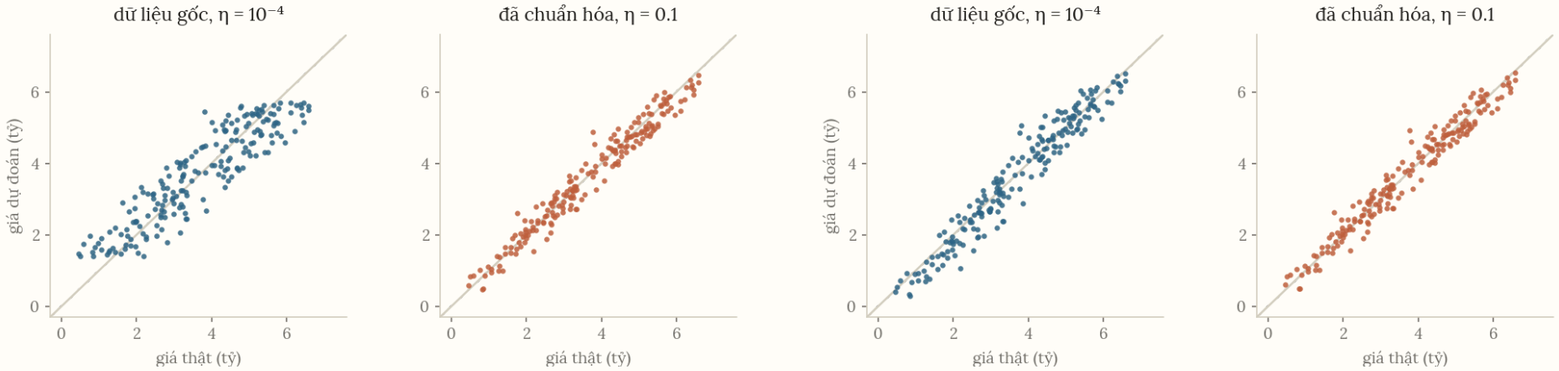
Đặc trưng chênh thang đo làm mặt loss thành thung lũng hẹp; chuẩn hóa đưa nó về gần tròn.

Cùng gradient descent, hai thang đo



vòng 1: MSE gốc 2.87, đã chuẩn hóa 10.2

vòng 5: MSE gốc 0.451, đã chuẩn hóa 1.77



vòng 20: MSE gốc 0.41, đã chuẩn hóa 0.0901

vòng 1000: MSE gốc 0.154, đã chuẩn hóa 0.0881

Cùng gradient descent trên 200 căn hộ: dữ liệu gốc tiến nhanh vài vòng đầu rồi gần như đứng yên; dữ liệu đã chuẩn hóa về sát đường chéo sau khoảng 20 vòng.

HỎI NHANH

Chuẩn hóa feature rồi học lại cho tới khi MSE chạm đáy. So với mô hình học trên dữ liệu gốc, cái gì thay đổi?

A Dự đoán chính xác hơn, vì thang đo tốt hơn

B Trọng số giữ nguyên, chỉ bias đổi theo trung bình

C Dự đoán y hệt, chỉ cách viết trọng số và tốc độ của gradient descent đổi

D `LinearRegression` của scikit-learn cũng hội tụ nhanh hơn

06

Thực hành và đọc kết quả

Dùng scikit-learn

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=0)
model = LinearRegression().fit(X_tr, y_tr)
print(model.score(X_tr, y_tr),
      model.score(X_te, y_te))
```

KẾT QUẢ

```
0.9604123189943734 0.9689505848749018
```

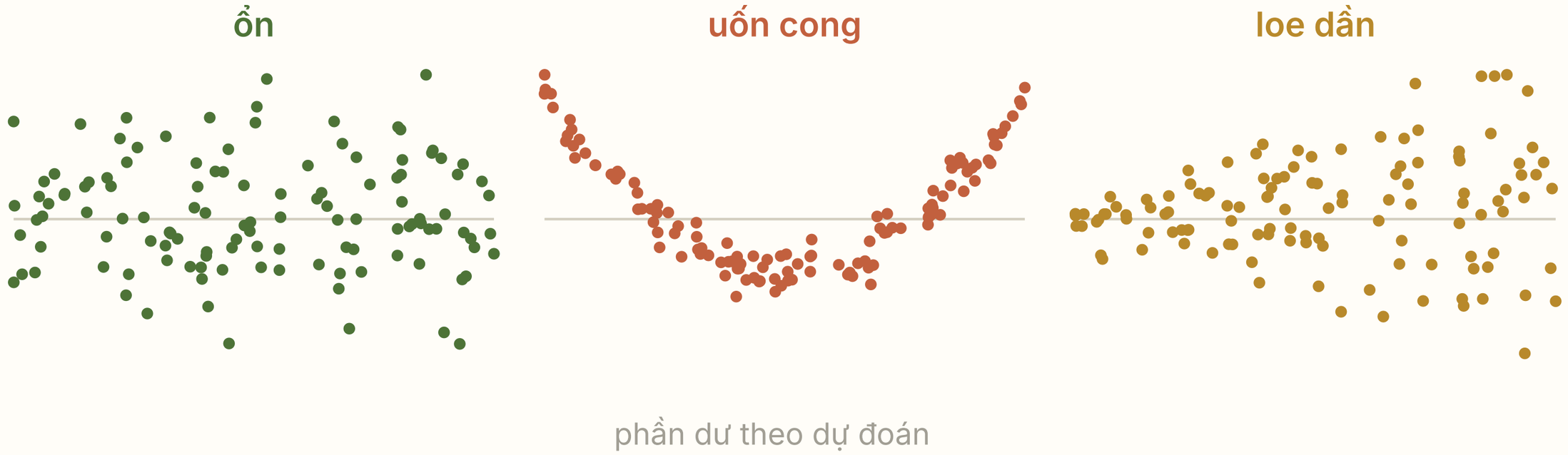
Kết quả trùng với code tự viết: `np.allclose` cho `True`.

Hai số lần lượt là R^2 train và test. Test cao hơn train có thể xảy ra do cách chia ngẫu nhiên, không tự là lỗi code.

Đọc hệ số cho đúng:

- Hệ số phụ thuộc đơn vị; muốn so sánh thì chuẩn hóa trước.
- Hệ số là mối liên hệ khi giữ các biến khác cố định, **không phải nhân quả**.
- Luôn vẽ phần dư theo dự đoán.

Đồ thị phần dư



Phần dư tốt nằm đều quanh 0; uốn cong báo thiếu quan hệ phi tuyến, loe dần báo nhiễu không đều.

HỎI NHANH

Linear regression cho hệ số dương giữa số kem bán ra và số vụ đuối nước mỗi tuần. Chỉ từ kết quả này, ta kết luận được gì?

A Giảm bán kem sẽ giảm số vụ đuối nước đúng theo hệ số

B Ăn kem làm tăng nguy cơ đuối nước khi bơi

C Nhiệt độ chắc chắn là nguyên nhân duy nhất của cả hai

D Hai đại lượng cùng tăng, còn tác động của kem thì chưa biết

Bẫy: broadcasting âm thầm

```
y_col = y.reshape(-1, 1)          # (200, 1)
y_hat = predict(X, w_cf, b_cf)    # (200,)
print((y_hat - y_col).shape)
print(mse(y_hat, y), mse(y_hat, y_col))
```

KẾT QUẢ

```
(200, 200)
0.08807615180405967 4.594878280120061
```

NumPy kéo cả hai thành `(200, 200)`: không báo lỗi, loss trông hợp lý nhưng vô nghĩa.

- Đọc nhãn bằng `df["price"]`, không dùng `df[["price"]]`.
- `assert y_hat.shape == y.shape` ở đầu hàm loss.

HỎI NHANH

`y_hat` có shape `(5, 1)`, `y` có shape `(5,)`. `y_hat - y` cho ra gì?

A Báo lỗi, vì hai shape khác nhau

B Mảng shape `(5,)`

C Mảng shape `(5, 1)`

D Mảng shape `(5, 5)`

Đoán độ phức tạp trước khi nộp

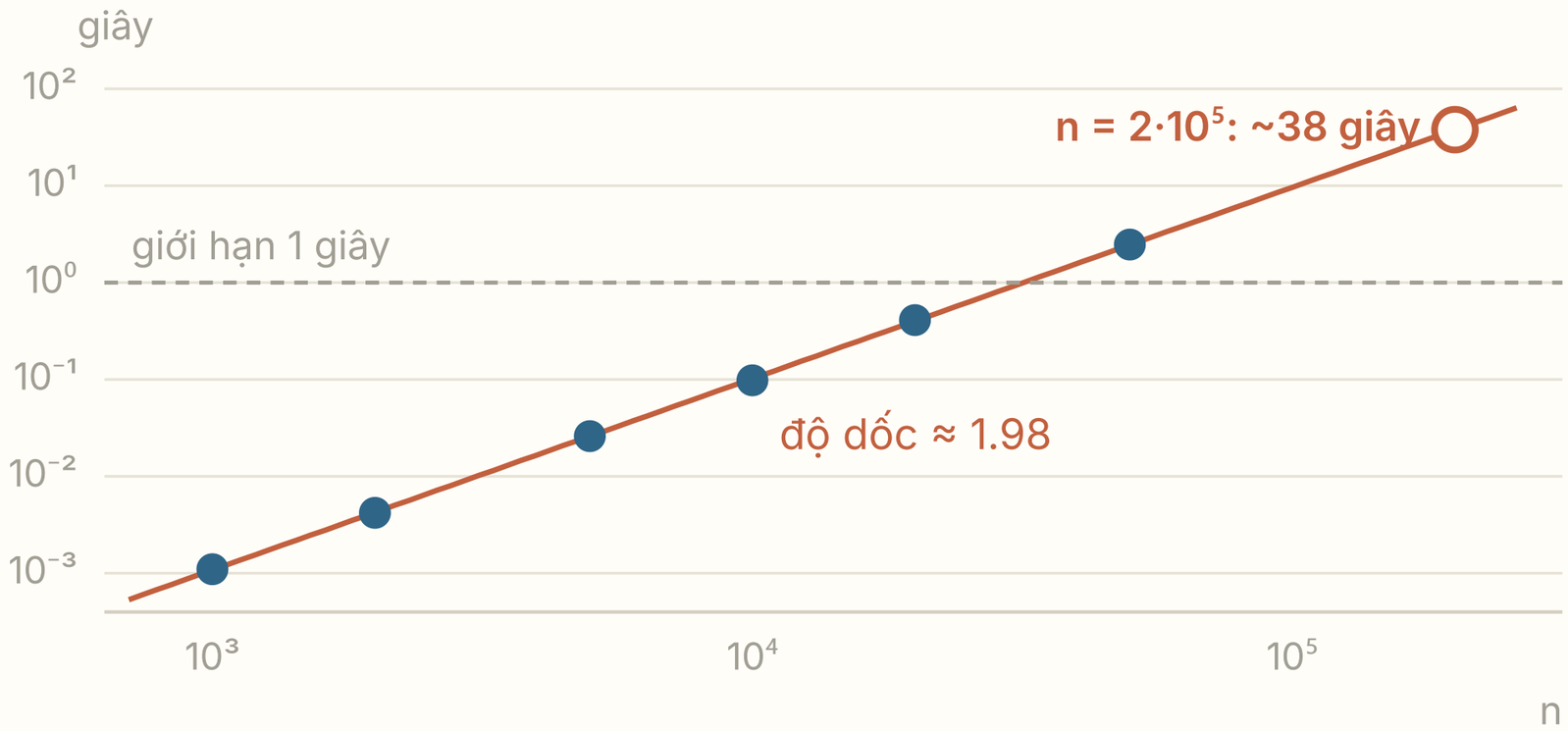
“Tuyến tính” là theo tham số; đặc trưng có thể là x^2, $\log x$, $x_1 x_2$.

Nếu $t \approx C n^k$ thì

$$\log t = k \log n + \log C$$

Hồi quy trên trục log–log: hệ số góc ≈ 1.98, gợi tăng gần bậc hai trong miền đo. Với $n = 2 \cdot 10^5$: ngoại suy khoảng 38 giây; chưa là chứng minh Big-O.

Độ dốc trên trục log–log



Trên trục log–log, thời gian theo n là đường thẳng; độ dốc gần 2 nghĩa là O(n²).

HỎI NHANH

Sau khi tạo thêm feature, mô hình nào **không** giải được bằng code linear regression của bài này?

A $\hat{y} = w_1 x + w_2 x^2 + b$

B $\hat{y} = w_1 \log x + b$

C $\hat{y} = w_1 x_1 x_2 + w_2 x_1 + b$

D $\hat{y} = e^{wx} + b$

Khi đi thi

- **Baseline cho hồi quy**: thử `LinearRegression` trên feature số và so với mốc hằng số.
- **Loss nổ**: chuẩn hóa đặc trưng trước, chỉnh learning rate sau.
- **Target không âm, metric RMSLE**: cân nhắc $\log(1 + y)$; kiểm miền prediction và quy ước scorer.
- **Ngoại suy**: mô hình tuyến tính kéo xu hướng ra ngoài vùng dữ liệu, mô hình cây thì không.
- **Tự viết thì tự kiểm**: kiểm shape và gradient trên dữ liệu nhỏ trước khi chạy thật.

Tóm tắt

- $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$, học bằng cách cực tiểu MSE; R^2 so với mốc đoán trung bình.
- Nghiệm đóng: giải $\tilde{X}^\top \tilde{X} \boldsymbol{\theta} = \tilde{X}^\top \mathbf{y}$ bằng `solve`; về hình học là phép chiếu vuông góc.
- Gradient descent hội tụ khi $\eta < 2/\lambda_{\max}$; chuẩn hóa đặc trưng kéo số điều kiện về gần 1.
- Dùng scikit-learn với train/test; hệ số là liên hệ, không phải nhân quả.

07

Luyện đề

LUYỆN ĐỀ 1

Với $L(\theta) = N^{-1} \sum_{n=0}^{N-1} (y_n - \theta^\top x_n)^2$, gradient theo θ là gì?

A $\frac{2}{N} \sum (\theta^\top x_n - y_n)$

B $\frac{2}{N} \sum (\theta^\top x_n - y_n) x_n$

C $\frac{2}{N} \sum (y_n - \theta^\top x_n) x_n$

D $\frac{1}{N} \sum (\theta^\top x_n - y_n) x_n$

ĐỀ THI USA-NA-AIO 2026 · Vòng 1 · Part 4.2

LUYỆN ĐỀ 2

Công thức nghiệm $\boldsymbol{\theta}^* = (\tilde{X}^\top \tilde{X})^{-1} \tilde{X}^\top \mathbf{y}$ dùng được khi nào?

A Luôn luôn, với mọi ma trận dữ liệu

B Khi $\tilde{X}$ vuông, tức số mẫu bằng số cột

C Khi các cột của $\tilde{X}$ độc lập tuyến tính

D Khi $\tilde{X}$ đối xứng qua đường chéo

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 34

LUYỆN ĐỀ 3

Hệ phương trình chuẩn của một bộ dữ liệu có số điều kiện cỡ 10^{10}. Vì sao điều này đáng lo khi giải bằng máy?

A Ma trận chắc chắn suy biến, nên hệ vô nghiệm

B Định thức cỡ 10^{10} nên phép tính bị tràn số

C Ma trận có quá nhiều phần tử khác 0 nên chạy chậm

D Sai số nhỏ ở dữ liệu có thể thành sai số lớn ở nghiệm

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 35

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook của bài: đổi learning rate, thử bỏ bước chuẩn hóa, và đọc lại kết quả.